Lab 6: Covert Channels in Image Metadata

0. Prepare the environment

```
from pathlib import Path
from PIL import Image, ExifTags

image_dir = Path("Images")
print("Images found:", len(list(image_dir.glob("*.png"))))
```

```
Images found: 6
```

The environment was successfully prepared, and 6 images were found.

1. First technique: EXIF metadata (one image)

```
image_path = image_dir / "Brain-Channel.png"
with Image.open(image_path) as img:
    exif = img.getexif()
    print("EXIF tags:", len(exif))
    for tag_id, value in exif.items():
        print(ExifTags.TAGS.get(tag_id, tag_id), ":", value)
```

```
EXIF tags: 5
ResolutionUnit : 2
ImageDescription : LATERAL: Pivot to internal Domain Controller using Mimikatz.
YCbCrPositioning : 1
XResolution : 72.0
YResolution : 72.0
```

Observe: Which EXIF field carries a message? Record it in your evidence log.

ImageDescription

Try it yourself: Change image_path to a different image. If EXIF tags: 0 appears, does that mean the image has no metadata? Explain.

```
# Your turn: choose another image and repeat the EXIF check.
image_path = image_dir / "Activation.png"
with Image.open(image_path) as img:
    exif = img.getexif()
    print("EXIF tags:", len(exif))
    for tag_id, value in exif.items():
        print(ExifTags.TAGS.get(tag_id, tag_id), ":", value)
# TODO: reuse the short EXIF snippet above
```

```
EXIF tags: 10
GPSInfo : 396
ResolutionUnit : 2
ExifOffset : 204
Make : Nikon
Model : D750
Software : Adobe Lightroom 13.0
DateTime : 2025:02:02 10:15:00
YCbCrPositioning : 1
XResolution : 72.0
YResolution : 72.0
```

No, it just means the image lacks standard EXIF metadata; hidden information could still exist in other metadata types like PNG text or XMP.

2. Second technique: PNG text metadata (one image)

```
image_path = image_dir / "Brain.png"
with Image.open(image_path) as img:
    for key, value in img.info.items():
        if isinstance(value, str):
            print(key, ":", value)
```

```
Software : RECON: Perform DNS harvesting and Subdomain enumeration.
```

Observe: Which PNG text field contains a clue? Try **Chemicals.png** next using the same short snippet.

Software

```
image_path = image_dir / "Chemicals.png"
with Image.open(image_path) as img:
    for key, value in img.info.items():
        if isinstance(value, str):
            print(key, ":", value)
# TODO: reuse the PNG text snippet above
```

```
ImageDescription : EXPLOIT: Deploy SQL Injection payload to bypass WAF.
```

3. Third technique: XMP metadata (one image)

```
image_path = image_dir / "Dopamine.png"
with Image.open(image_path) as img:
    xmp = img.info.get("XML:com.adobe.xmp") or img.info.get("xmp")
    if isinstance(xmp, bytes):
        xmp = xmp.decode("utf-8", errors="replace")
    print(xmp if xmp else "No XMP found")
```

```
<?xpacket begin='' id='W5M0MpCehiHzreSzNTczkc9d'?>
<x:xmpmeta xmlns:x='adobe:ns:meta/' x:xmptk='Image::ExifTool 12.76'>
<rdf:RDF xmlns:rdf='http://www.w3.org/1999/02/22-rdf-syntax-ns#'>

 <rdf:Description rdf:about=''
  xmlns:exif='http://ns.adobe.com/exif/1.0/'>
  <exif:MakerNote>PERSIST: Establish reverse shell via obfuscated cron job.</exif:MakerNote>
 </rdf:Description>
</rdf:RDF>
</x:xmpmeta>
<?xpacket end='r'?>
```

Observe: Locate the XML element containing the partial secret. Why did the EXIF-only approach miss it?

<exif:MakerNote>. The EXIF-only approach missed it because the data is embedded inside an XML-based **XMP metadata block**, which standard EXIF functions like img.getexif() cannot extract.

4. Fourth technique: Decode suspicious text (one image)

```
encoded_message = "RVhGSUw6IFR1bm5lbCBjb21wcmVzc2VkIGRhdGFiYXNlIGR1bXAgdmlhIERORU yBDb3ZlcnQgQ2hhbm5lbC4="
if encoded_message:
    print(base64.b64decode(encoded_message).decode("utf-8"))
```

```
EXFIL: Tunnel compressed database dump via DNS Covert Channel.
```

EXIF metadata.

Your investigation: six images, one by one

Investigation 1: Activation.png

```
image_path = image_dir / "Activation.png"
# TODO: paste or adapt the relevant short snippet from Sections 1-4
with Image.open(image_path) as img:
    exif = img.getexif()
    print("EXIF tags:", len(exif))
    for tag_id, value in exif.items():
        print(ExifTags.TAGS.get(tag_id, tag_id), ":", value)
```

```
EXIF tags: 10
GPSInfo : 396
ResolutionUnit : 2
ExifOffset : 204
Make : Nikon
Model : D750
Software : Adobe Lightroom 13.0
DateTime : 2025:02:02 10:15:00
YCbCrPositioning : 1
XResolution : 72.0
YResolution : 72.0
```

```
with Image.open(image_path) as img:
    for key, value in img.info.items():
        if isinstance(value, str):
            print(key, ":", value)
```

```
Make : Nikon
Model : D750
Software : Adobe Lightroom 13.0
create-date : 2025-02-02T10:15:00
```

```
with Image.open(image_path) as img:
    xmp = img.info.get("XML:com.adobe.xmp") or img.info.get("xmp")
    if isinstance(xmp, bytes):
        xmp = xmp.decode("utf-8", errors="replace")
    print(xmp if xmp else "No XMP found")
```

```
No XMP found
```

```
with Image.open(image_path) as img:
    exif_bytes = img.info.get("exif", b"")
    exif_text = exif_bytes.decode("utf-8", errors="ignore")
    print(exif_text)
```

```
ExifMM*
□□□□□□□□□□□□□□□□□□(□□□□1□□□2□□□□□□□i□□%□□□NikonD750H□H□Adobe Lightroom 13.02025:02:02 10:15:00□□□□0232□□□□&□□□□:□□□□□□□>□N□□0100□□□2025:02:02 10:15:00202
5:02:02 10:15:00ASCIIVULN: Identify unpatched CVE-2024-XXXX in web gateway.□□□□□□□□N□□□□□□□W□□□□(□*□□□J□□l□
```

Investigation 2: Brain-Channel.png

Investigation 2: `Brain-Channel.png`

Choose and reuse the appropriate short extraction snippet(s) above. Record the evidence in your table. **What is the partial secret?**

```
image_path = image_dir / "Brain-Channel.png"
# TODO: paste or adapt the relevant short snippet from Sections 1-4
with Image.open(image_path) as img:
    exif = img.getexif()
    print("EXIF tags:", len(exif))
    for tag_id, value in exif.items():
        print(ExifTags.TAGS.get(tag_id, tag_id), ":", value)
```

```
EXIF tags: 5
ResolutionUnit : 2
ImageDescription : LATERAL: Pivot to internal Domain Controller using Mimikatz.
YCbCrPositioning : 1
XResolution : 72.0
YResolution : 72.0
```

Investigation 3: Brain.png

Investigation 3: `Brain.png`

Choose and reuse the appropriate short extraction snippet(s) above. Record the evidence in your table. **What is the partial secret?**

```
image_path = image_dir / "Brain.png"
# TODO: paste or adapt the relevant short snippet from Sections 1-4
with Image.open(image_path) as img:
    exif = img.getexif()
    print("EXIF tags:", len(exif))
    for tag_id, value in exif.items():
        print(ExifTags.TAGS.get(tag_id, tag_id), ":", value)

with Image.open(image_path) as img:
    for key, value in img.info.items():
        if isinstance(value, str):
            print(key, ":", value)
```

```
EXIF tags: 0
Software : RECON: Perform DNS harvesting and Subdomain enumeration.
```

Investigation 4: Chemicals.png

Investigation 4: `Chemicals.png`

Choose and reuse the appropriate short extraction snippet(s) above. Record the evidence in your table. **What is the partial secret?**

```
image_path = image_dir / "Chemicals.png"
# TODO: paste or adapt the relevant short snippet from Sections 1-4
with Image.open(image_path) as img:
    exif = img.getexif()
    print("EXIF tags:", len(exif))
    for tag_id, value in exif.items():
        print(ExifTags.TAGS.get(tag_id, tag_id), ":", value)

with Image.open(image_path) as img:
    for key, value in img.info.items():
        if isinstance(value, str):
            print(key, ":", value)
```

```
EXIF tags: 0
ImageDescription : EXPLOIT: Deploy SQL Injection payload to bypass WAF.
```

Investigation 5: Dopamine.png

Investigation 5: `Dopamine.png` ¶

Choose and reuse the appropriate short extraction snippet(s) above. Record the evidence in your table. **What is the partial secret?**

```
image_path = image_dir / "Dopamine.png"
# TODO: paste or adapt the relevant short snippet from Sections 1-4
with Image.open(image_path) as img:
    exif = img.getexif()
    print("EXIF tags:", len(exif))
    for tag_id, value in exif.items():
        print(ExifTags.TAGS.get(tag_id, tag_id), ":", value)

with Image.open(image_path) as img:
    for key, value in img.info.items():
        if isinstance(value, str):
            print(key, ":", value)

with Image.open(image_path) as img:
    xmp = img.info.get("XML:com.adobe.xmp") or img.info.get("xmp")
    if isinstance(xmp, bytes):
        xmp = xmp.decode("utf-8", errors="replace")
    print(xmp if xmp else "No XMP found")
```

```
EXIF tags: 0
XML:com.adobe.xmp : <?xpacket begin='' id='W5M0MpCehiHzreSzNTczkc9d'?>
<x:xmpmeta xmlns:x='adobe:ns:meta/' x:xmptk='Image::ExifTool 12.76'>
<rdf:RDF xmlns:rdf='http://www.w3.org/1999/02/22-rdf-syntax-ns#'>

 <rdf:Description rdf:about=''
  xmlns:exif='http://ns.adobe.com/exif/1.0/'>
  <exif:MakerNote>PERSIST: Establish reverse shell via obfuscated cron job.</exif:MakerNote>
 </rdf:Description>
</rdf:RDF>
</x:xmpmeta>
<?xpacket end='r'?>
<?xpacket begin='' id='W5M0MpCehiHzreSzNTczkc9d'?>
<x:xmpmeta xmlns:x='adobe:ns:meta/' x:xmptk='Image::ExifTool 12.76'>
<rdf:RDF xmlns:rdf='http://www.w3.org/1999/02/22-rdf-syntax-ns#'>

 <rdf:Description rdf:about=''
  xmlns:exif='http://ns.adobe.com/exif/1.0/'>
  <exif:MakerNote>PERSIST: Establish reverse shell via obfuscated cron job.</exif:MakerNote>
 </rdf:Description>
</rdf:RDF>
</x:xmpmeta>
<?xpacket end='r'?>
```

Investigation 6: Music.png

Investigation 6: `Music.png` ¶

Choose and reuse the appropriate short extraction snippet(s) above. Record the evidence in your table. **What is the partial secret?**

```
image_path = image_dir / "Music.png"
# TODO: paste or adapt the relevant short snippet from Sections 1-4
with Image.open(image_path) as img:
    exif = img.getexif()
    print("EXIF tags:", len(exif))
    for tag_id, value in exif.items():
        print(ExifTags.TAGS.get(tag_id, tag_id), ":", value)

encoded_message = "RVhGSUw6IFR1bm5lbCBjb21wcmVzc2VkIGRhdGFiYXNlIGR1bXAgdmlhIEROUyBDb3ZlcnQgQ2hhbm5lbC4="  # TODO: paste the en
if encoded_message:
    print(base64.b64decode(encoded_message).decode("utf-8"))
```

```
EXIF tags: 9
ResolutionUnit : 2
ExifOffset : 264
Make : Olympus
Model : OM-D E-M10
DateTime : 2025:02:06 11:11:00
YCbCrPositioning : 1
Copyright : RVhGSUw6IFR1bm5lbCBjb21wcmVzc2VkIGRhdGFiYXNlIGR1bXAgdmlhIEROUyBDb3ZlcnQgQ2hhbm5lbC4=
XResolution : 72.0
YResolution : 72.0
EXFIL: Tunnel compressed database dump via DNS Covert Channel.
```

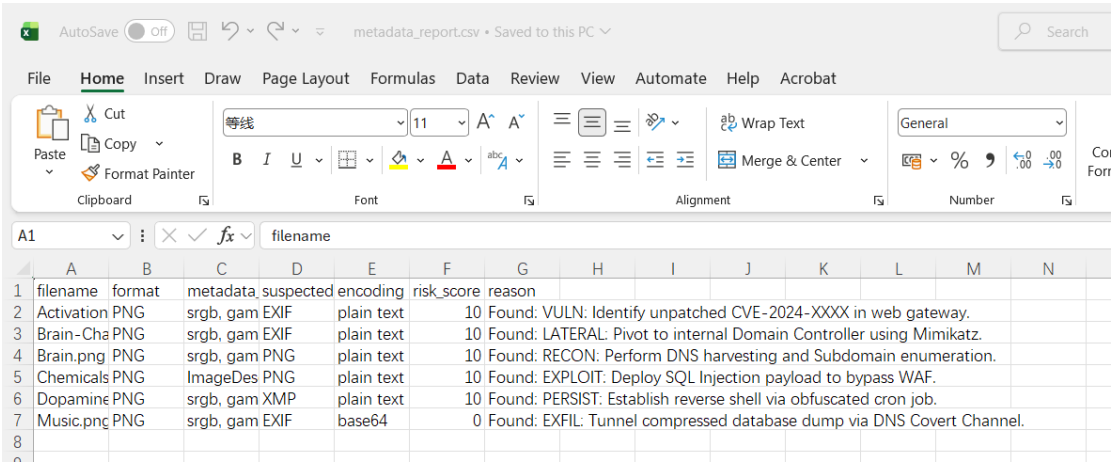
Image	Metadata type / field	Decoding needed?	Partial secret recovered
Activation.png	EXIF: Raw bytes	No	VULN: Identify unpatched CVE-2024-XXXX in web gateway.
Brain-Channel.png	EXIF: ImageDescription	No	LATERAL: Pivot to internal Domain Controller using Mimikatz.
Brain.png	PNG text: Software	No	RECON: Perform DNS harvesting and Subdomain enumeration.
Chemicals.png	PNG text: ImageDescription	No	EXPLOIT: Deploy SQL Injection payload to bypass WAF.
Dopamine.png	XMP: exif:MakerNote	No	PERSIST: Establish reverse shell via obfuscated cron job.
Music.png	EXIF: Copyright	Yes	EXFIL: Tunnel compressed database dump via DNS Covert Channel

6. Reconstruct the complete secret

1. **Stage:** RECON **Recovered message:** Perform DNS harvesting and Subdomain enumeration.
2. **Stage:** VULN **Recovered message:** Identify unpatched CVE-2024-XXXX in web gateway.
3. **Stage:** EXPLOIT **Recovered message:** Deploy SQL Injection payload to bypass WAF.
4. **Stage:** PERSIST **Recovered message:** Establish reverse shell via obfuscated cron job.
5. **Stage:** LATERAL **Recovered message:** Pivot to internal Domain Controller using Mimikatz.
6. **Stage:** EXFIL **Recovered message:** Tunnel compressed database dump via DNS Covert Channel.

This sequence follows the natural steps of a cyberattack: attackers first gather information (RECON), find a weakness (VULN), and break in (EXPLOIT). Once inside, they set up access to stay (PERSIST), move around the network (LATERAL), and finally steal the data (EXFIL).

7. Build an automated metadata scanner



	A	B	C	D	E	F	G
1	filename	format	metadata	suspected	encoding	risk_score	reason
2	Activation	PNG	srgb, gam	EXIF	plain text	10	Found: VULN: Identify unpatched CVE-2024-XXXX in web gateway.
3	Brain-Cha	PNG	srgb, gam	EXIF	plain text	10	Found: LATERAL: Pivot to internal Domain Controller using Mimikatz.
4	Brain.png	PNG	srgb, gam	PNG	plain text	10	Found: RECON: Perform DNS harvesting and Subdomain enumeration.
5	Chemicals	PNG	ImageDes	PNG	plain text	10	Found: EXPLOIT: Deploy SQL Injection payload to bypass WAF.
6	Dopamine	PNG	srgb, gam	XMP	plain text	10	Found: PERSIST: Establish reverse shell via obfuscated cron job.
7	Music.png	PNG	srgb, gam	EXIF	base64	0	Found: EXFIL: Tunnel compressed database dump via DNS Covert Channel.
8							

1. Why can an image contain a hidden message even when getexif() returns no tags?

   EXIF is only one metadata format; hidden data can reside in PNG text chunks, XMP, trailing bytes, or steganography.

2. How is **encoding** different from **encryption**? Is Base64 a secure way to hide information?

   Encoding changes data format without a key, whereas encryption secures it with a key. Base64 is not secure and can be easily decoded by anyone.

3. Why should investigators preserve the original files and record hashes?

   To maintain evidence integrity, prove files haven't been tampered with, and ensure chain of custody.

4. Can metadata alone prove that an image was edited or that an incident occurred? What corroborating evidence would you need?

   Metadata can be easily faked or stripped, so you need manual analysis, system logs, or witness statements to corroborate it.

5. How does this investigation connect to the automated evidence collection and analysis you practised in Labs 4 and 5?

   It applies automated scripting to extract, parse, and analyze digital evidence efficiently at scale.

Github

https://github.com/bnwzc/FSCT8561-Labs-Kuangda-Zhu